Terraform-04

1. **Watch Terraform-04 video.**

2. **Execute the script shown in the video.ss**

Version Constraints:
==========
Changing in terraform providers version may get us into incompatability issues.
By default terraform will always try to downlaod the latest version of provider available from registry.

To make sure to use the specific version provider we can add the provider block in configuration.

Example:
=======

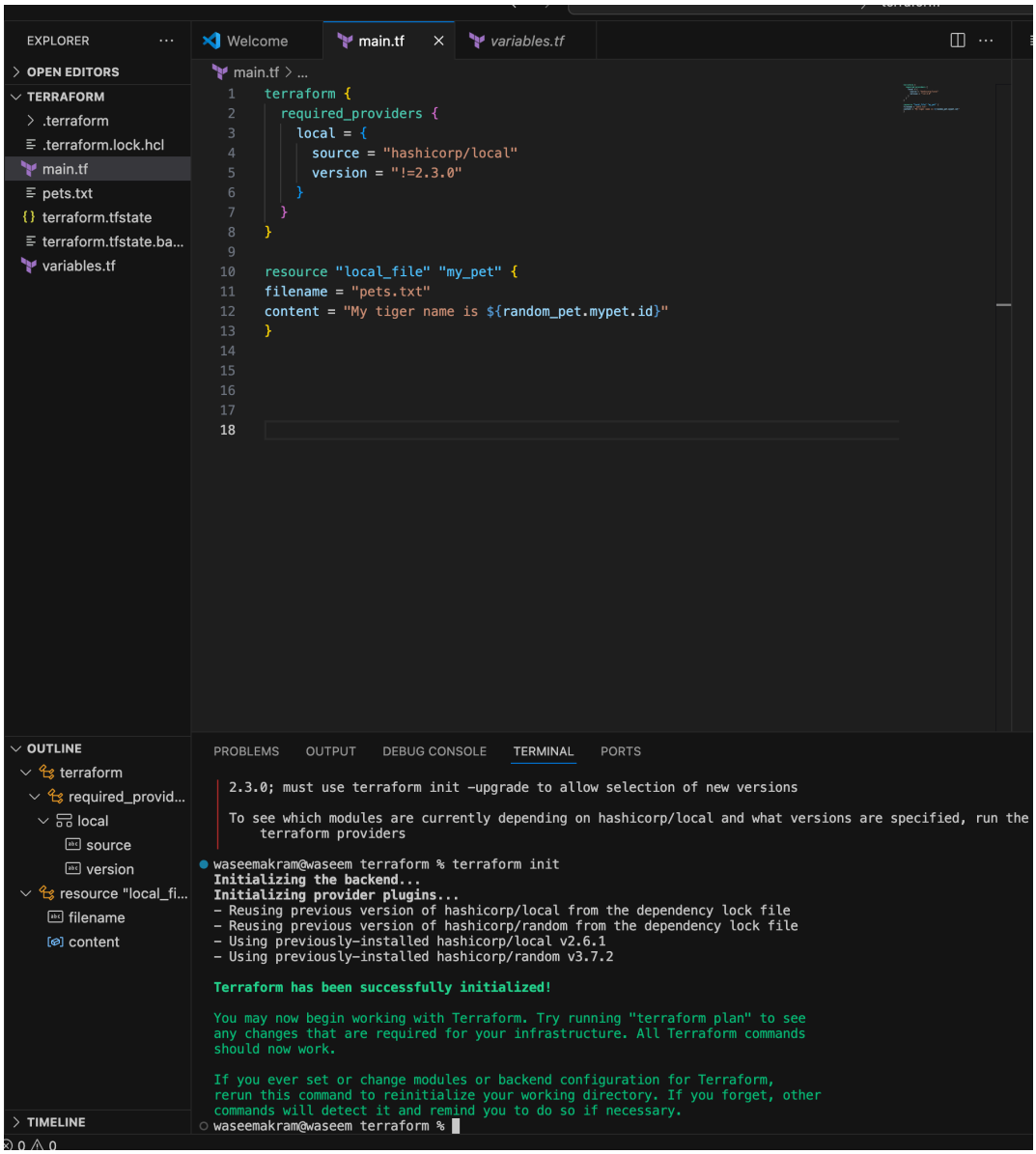
```
terraform {
 required_providers {
  local = {
   source = "hashicorp/local"
   version = "2.3.0"
  }
 }
}
```

```
waseemakram@waseem terraform % terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/local from the dependency lock file
- Reusing previous version of hashicorp/random from the dependency lock file
- Using previously-installed hashicorp/random v3.7.2

Error: Failed to query available provider packages

Could not retrieve the list of available versions for provider hashicorp/local: locked provider registry.terraform.io/hashicorp/local 2.6.1 does not match configured ve
2.3.0; must use terraform init -upgrade to allow selection of new versions

To see which modules are currently depending on hashicorp/local and what versions are specified, run the following command:
    terraform providers
```

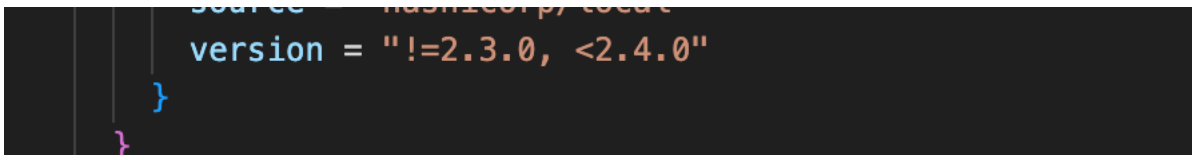
```
    terraform providers

waseemakram@waseem terraform % terraform init -upgrade
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/local versions matching "2.3.0"...
- Finding latest version of hashicorp/random...
- Installing hashicorp/local v2.3.0...
- Installed hashicorp/local v2.3.0 (signed by HashiCorp)
- Installing hashicorp/random v3.8.0...
- Installed hashicorp/random v3.8.0 (signed by HashiCorp)
Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
```

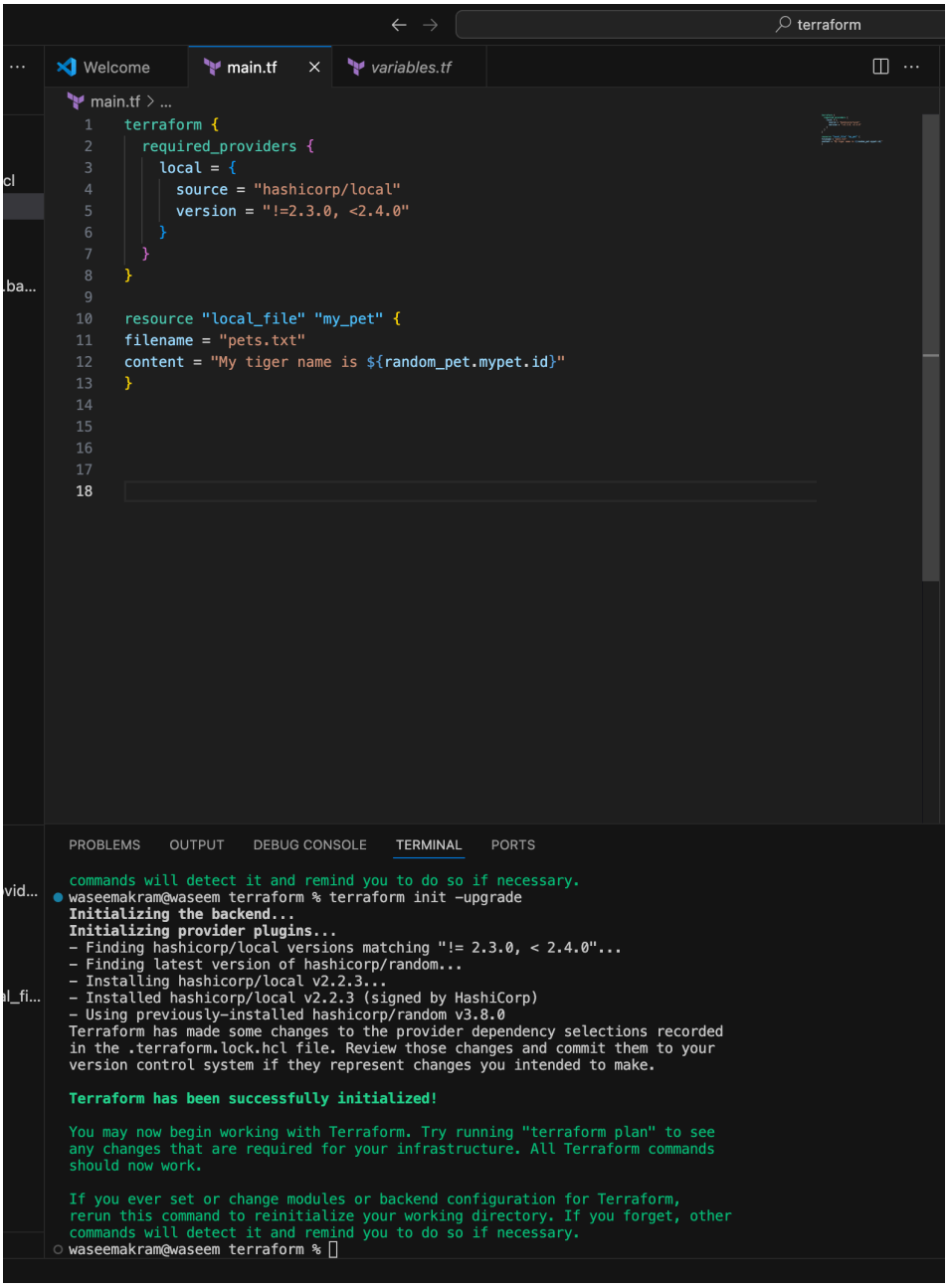
version = "2.3.0" --> download the exact version
version = "!=2.3.0" --> will not use the mentioned version
version = "< 2.3.0" --> lesses than the mention version
version = "> 2.3.0" --> greater than the given version
version = "~> 2.3.0" --> specific version or higher version.

—>


EXPLORER
OPEN EDITORS
TERRAFORM
.terraform
.terraform.lock.hcl
main.tf
pets.txt
terraform.tfstate
terraform.tfstate.ba...
variables.tf
Welcome
main.tf
variables.tf
main.tf > ...
terraform {
required_providers {
local = {
source = "hashicorp/local"
version = "!=2.3.0"
}
}
}
resource "local_file" "my_pet" {
filename = "pets.txt"
content = "My tiger name is ${random_pet.mypet.id}"
}
OUTLINE
terraform
required_provid...
local
source
version
resource "local_fi...
filename
content
TIMELINE
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
2.3.0; must use terraform init -upgrade to allow selection of new versions
To see which modules are currently depending on hashicorp/local and what versions are specified, run the terraform providers
waseemakram@waseem terraform % terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/local from the dependency lock file
- Reusing previous version of hashicorp/random from the dependency lock file
- Using previously-installed hashicorp/local v2.6.1
- Using previously-installed hashicorp/random v3.7.2
Terraform has been successfully initialized!
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
waseemakram@waseem terraform %

—>


version = "!=2.3.0, <2.4.0"
}
}



Data sources:
==========
Apart from terraform we have multiple other tools where the infra can be created.
Ex: ansible,salt,puppet,bash script,manual process.

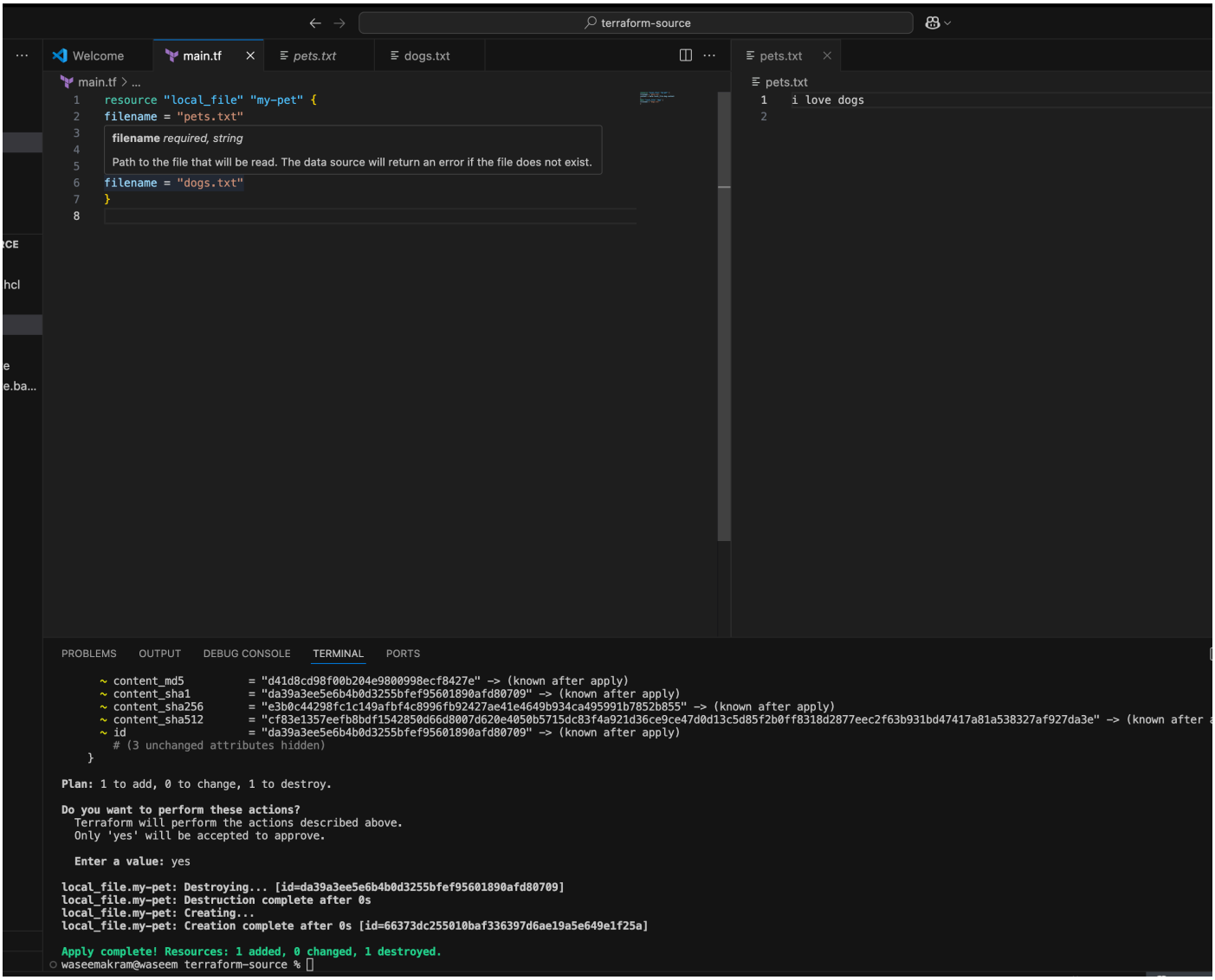
Data sources are used to read the content of the infrastructure

for example if we want terraform to read the content of the file which has been created by any other tool.

create a file called in dogs.txt in the same terrafrom working directory.

main.tf
======

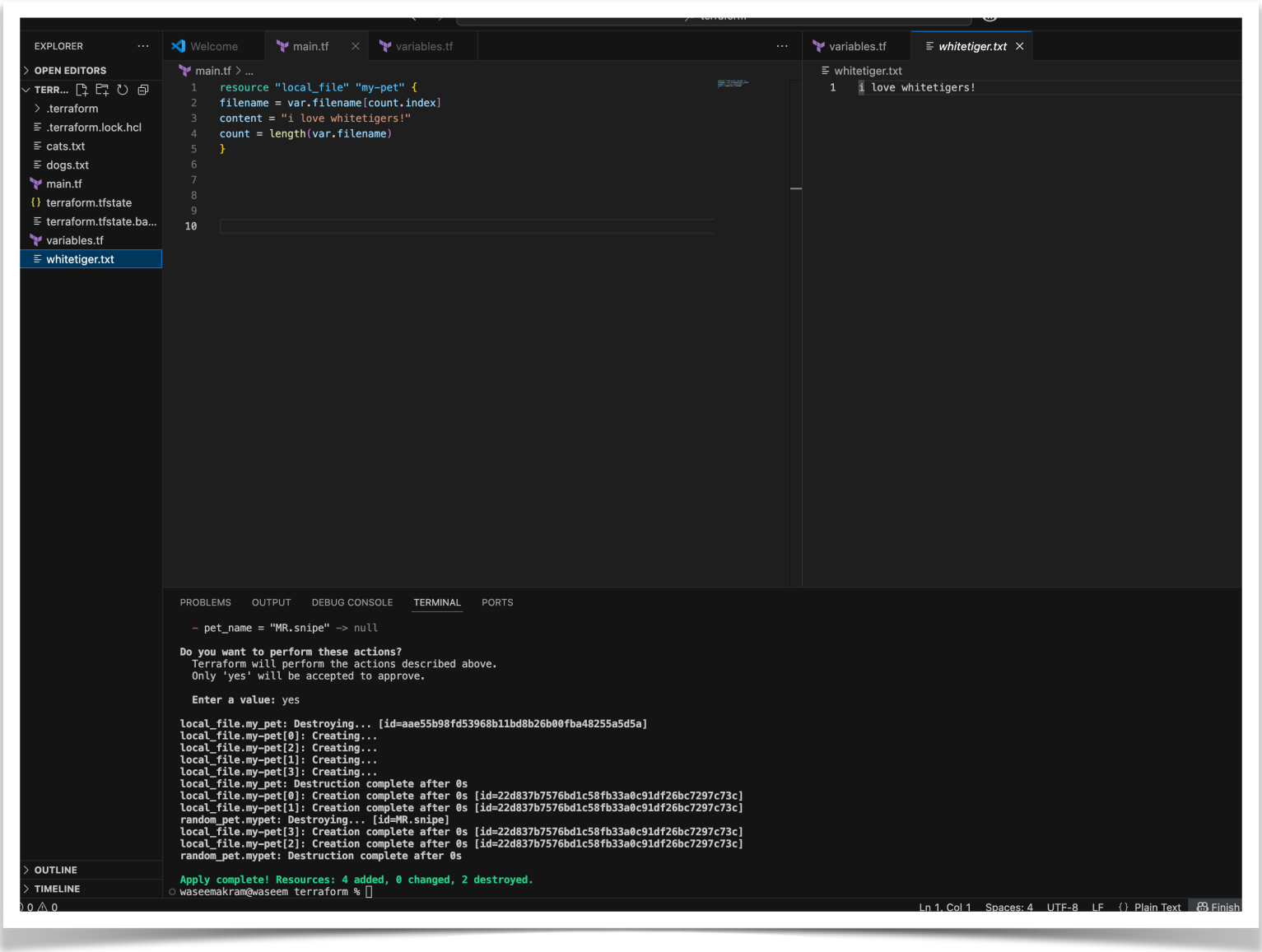
```
resource "local_file" "my-pet" {
filename = "pets.txt"
content = data.local_file.dog.content
}
data "local_file" "dog" {
filename = "dogs.txt"
}
```

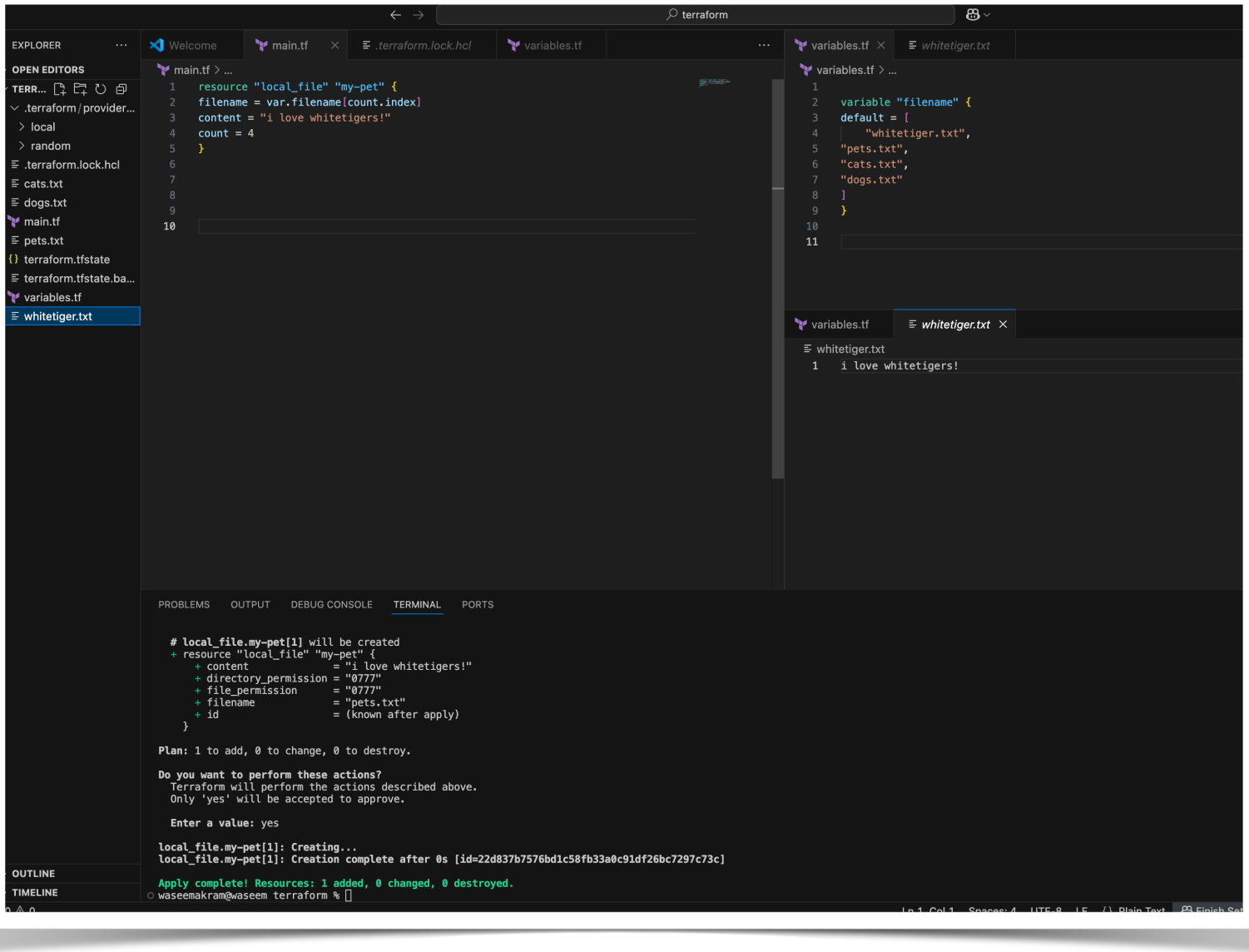


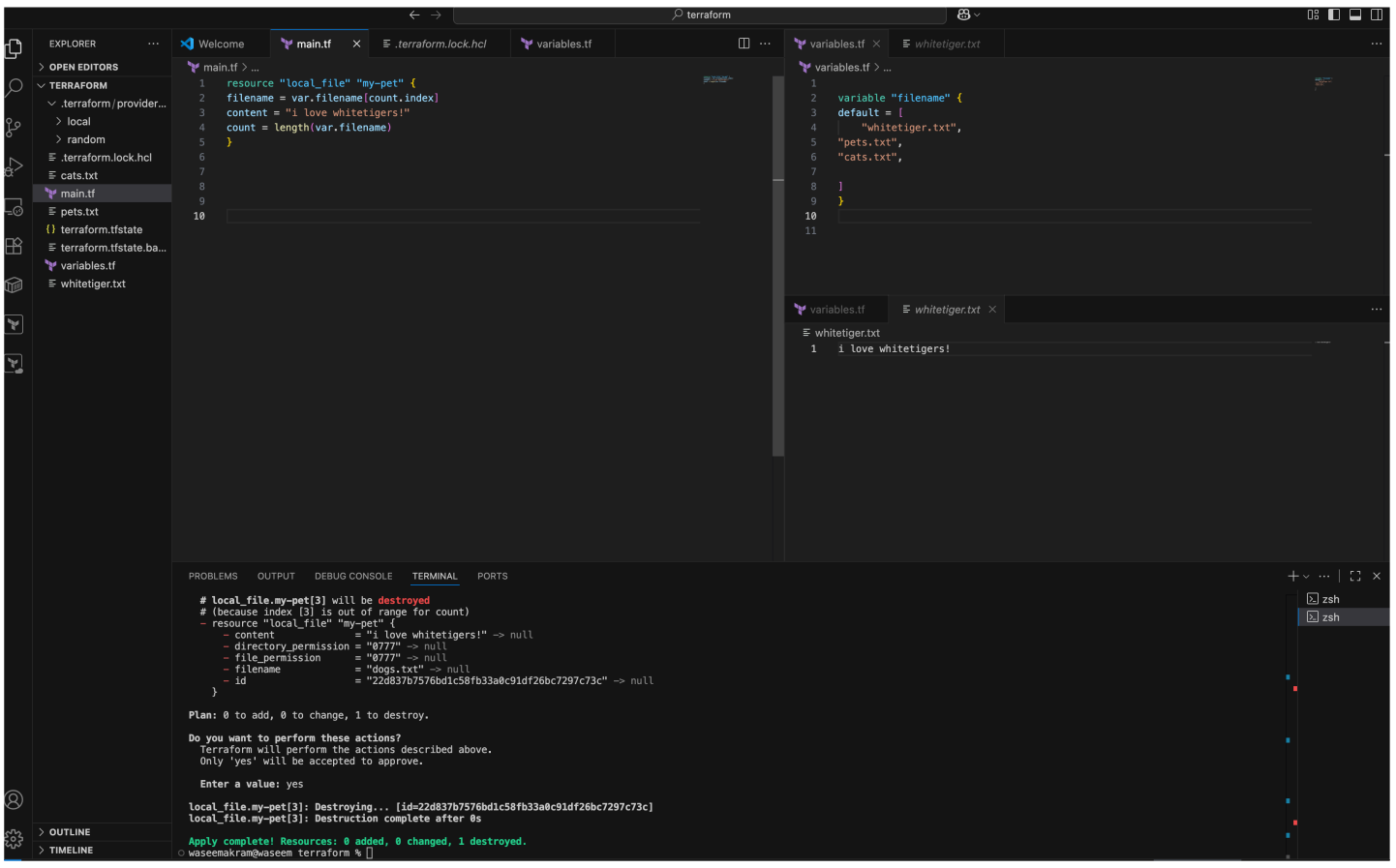
Meta-Arguments:
==========
Meta arguments are used if we want to create multiple resources.
Meta arguments can be used within any resource block to change the behaviour of the resources.
Examples for meta arguments:

1) Depends_on
2) lifecycle rules
3) Count
4) For_each

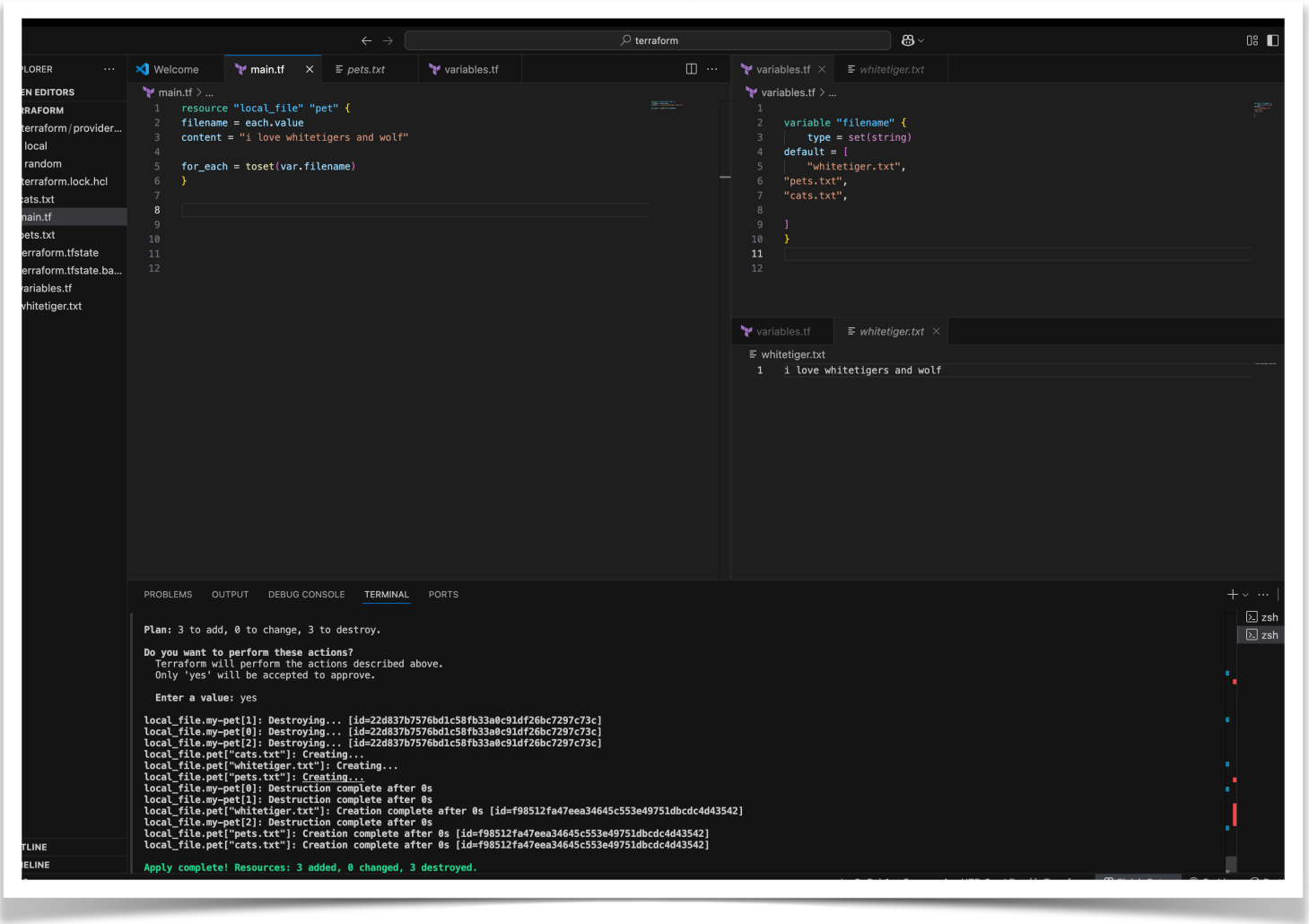
Example of count:
==========
If we use count as 3 then it will create 3 files with pet[0],pet[1],pet[2]



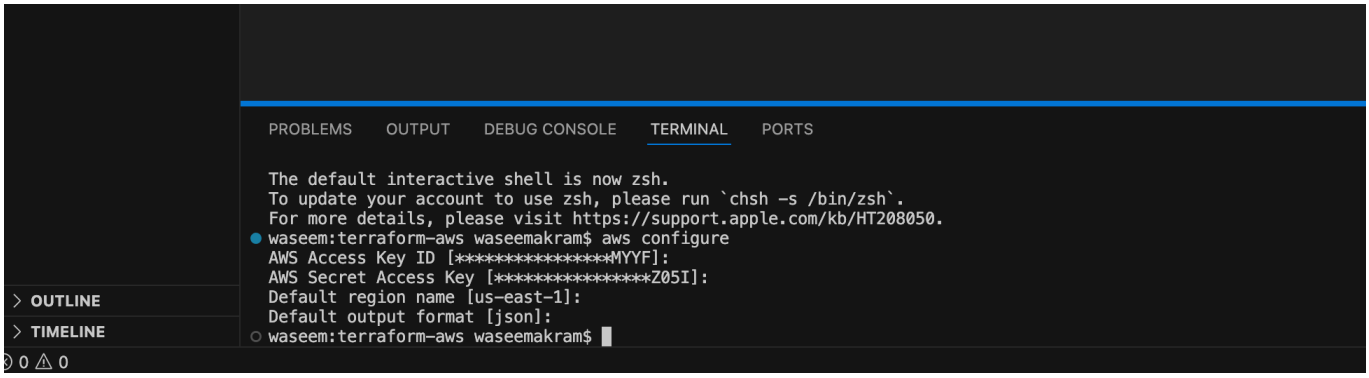

terraform
EXPLORER
OPEN EDITORS
TERR...
.terraform / provider...
local
random
.terraform.lock.hcl
cats.txt
dogs.txt
main.tf
pets.txt
terraform.tfstate
terraform.tfstate.ba...
variables.tf
whitetiger.txt
Welcome
main.tf
.terraform.lock.hcl
variables.tf
main.tf > ...
resource "local_file" "my-pet" {
filename = var.filename[count.index]
content = "i love whitetigers!"
count = 4
}
variables.tf
whitetiger.txt
variables.tf > ...
variable "filename" {
default = [
"whitetiger.txt",
"pets.txt",
"cats.txt",
"dogs.txt"
]
}
whitetiger.txt
i love whitetigers!
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
# local_file.my-pet[1] will be created
+ resource "local_file" "my-pet" {
+ content = "i love whitetigers!"
+ directory_permission = "0777"
+ file_permission = "0777"
+ filename = "pets.txt"
+ id = (known after apply)
}
Plan: 1 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
local_file.my-pet[1]: Creating...
local_file.my-pet[1]: Creation complete after 0s [id=22d837b7576bd1c58fb33a0c91df26bc7297c73c]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
waseemakram@waseem terraform %
OUTLINE
TIMELINE


terraform
EXPLORER
OPEN EDITORS
TERRAFORM
.terraform / provider...
local
random
.terraform.lock.hcl
cats.txt
main.tf
pets.txt
terraform.tfstate
terraform.tfstate.ba...
variables.tf
whitetiger.txt
Welcome
main.tf
.terraform.lock.hcl
variables.tf
main.tf > ...
resource "local_file" "my-pet" {
filename = var.filename[count.index]
content = "i love whitetigers!"
count = length(var.filename)
}
variables.tf
whitetiger.txt
variables.tf > ...
variable "filename" {
default = [
"whitetiger.txt",
"pets.txt",
"cats.txt",
]
}
whitetiger.txt
i love whitetigers!
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
zsh
zsh
# local_file.my-pet[3] will be destroyed
# (because index [3] is out of range for count)
- resource "local_file" "my-pet" {
- content = "i love whitetigers!" -> null
- directory_permission = "0777" -> null
- file_permission = "0777" -> null
- filename = "dogs.txt" -> null
- id = "22d837b7576bd1c58fb33a0c91df26bc7297c73c" -> null
}
Plan: 0 to add, 0 to change, 1 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
local_file.my-pet[3]: Destroying... [id=22d837b7576bd1c58fb33a0c91df26bc7297c73c]
local_file.my-pet[3]: Destruction complete after 0s
Apply complete! Resources: 0 added, 0 changed, 1 destroyed.
waseemakram@waseem terraform %
OUTLINE
TIMELINE

Example of for_each:
====================

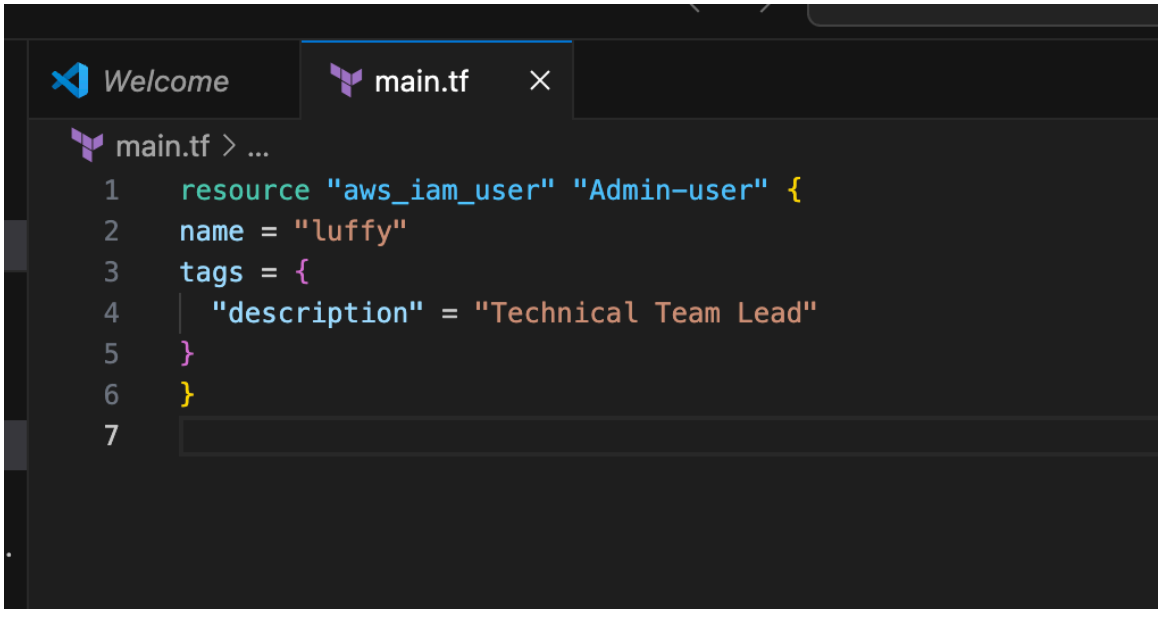


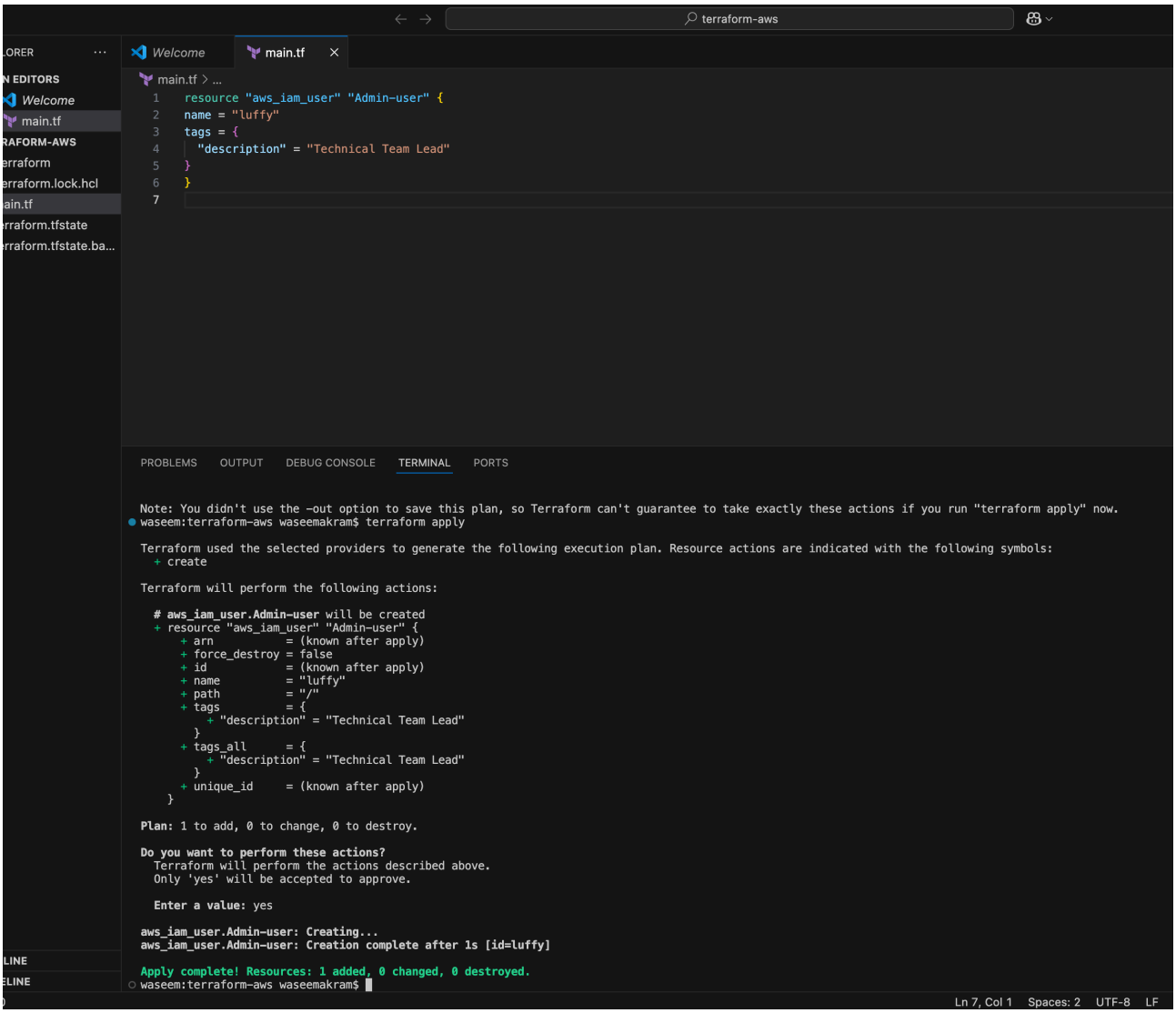
Terraform with AWS:
==========
--> first we need to create a secret key and access key and configure then in laptop.



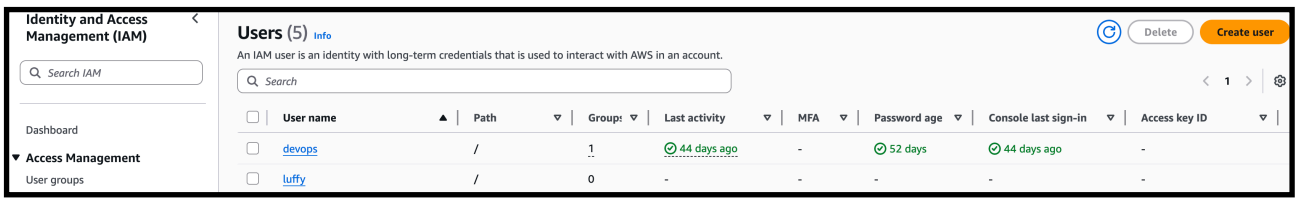
Example script to create aws iam:
==========

```
resource "aws_iam_user" "Admin-user" {
name = "lucy"
tags = {
  "description" = "Technical Team Lead"
}
}
```



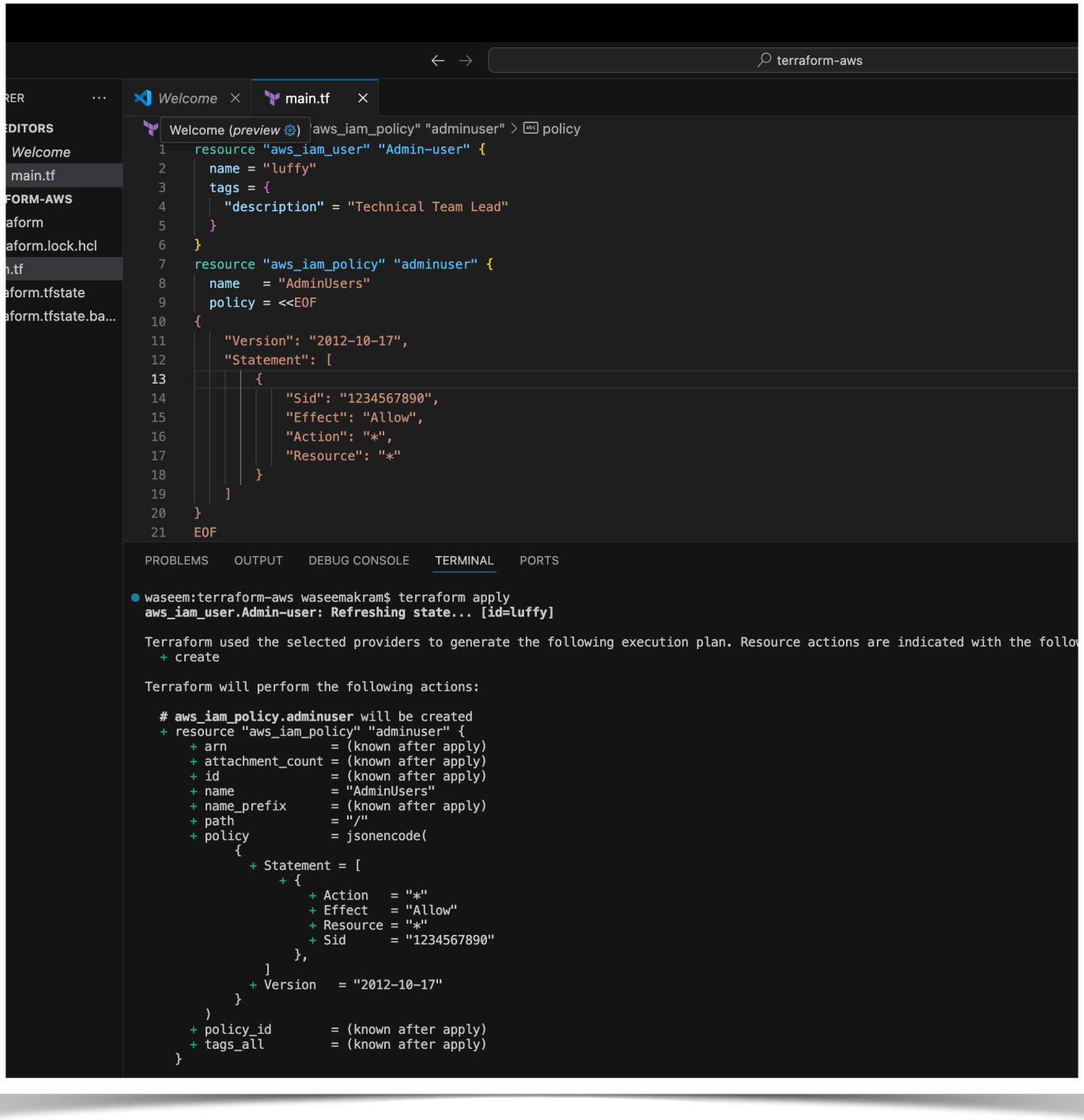


—-> now check your aws account and inside—-> iam check —->users
—> we will able to see puffy named user in Iam



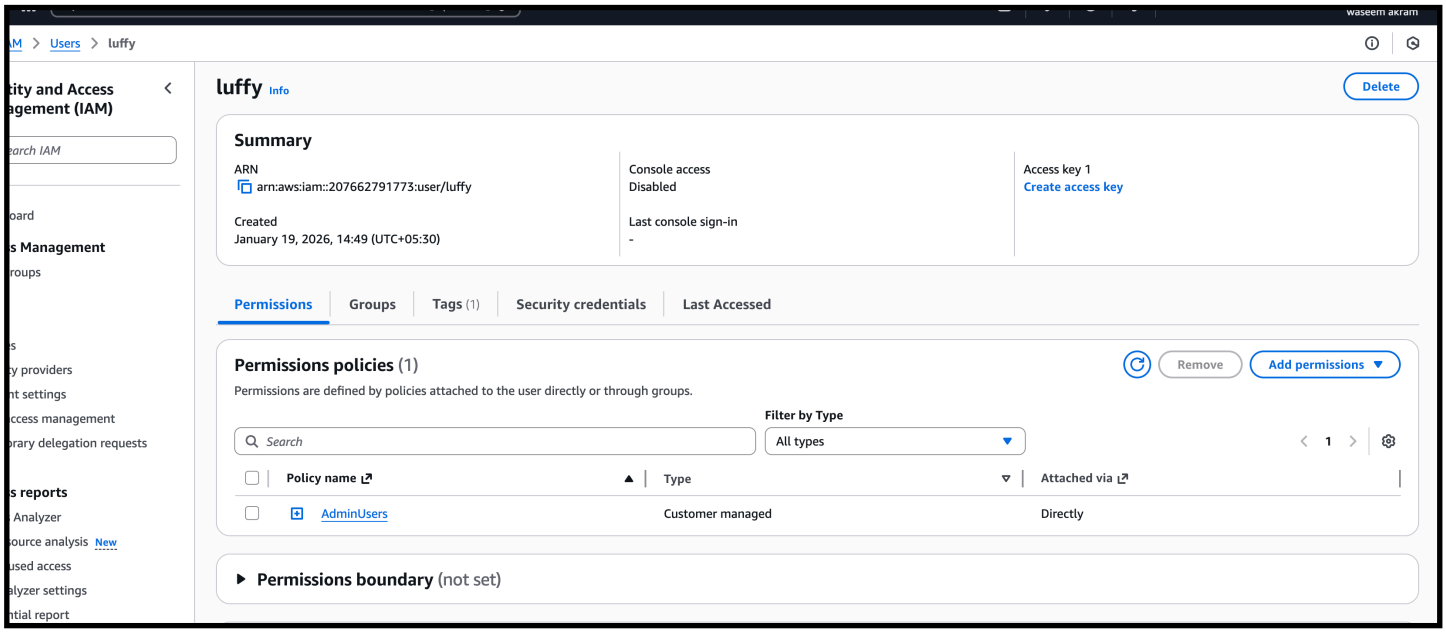
Policy attach to the user

```
resource "aws_iam_user" "Admin-user" {
  name = "lucy"
  tags = {
    "description" = "Technical Team Lead"
  }
}
resource "aws_iam_policy" "adminuser" {
  name   = "AdminUsers"
  policy = <<EOF
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "1234567890",
            "Effect": "Allow",
            "Action": "*",
            "Resource": "*"
        }
    ]
}
EOF
}

resource "aws_iam_user_policy_attachment" "lucy-admin-access"
{
  user       = aws_iam_user.Admin-user.name
  policy_arn = aws_iam_policy.adminuser.arn
}
```


terraform-aws
Welcome
main.tf
Welcome (preview)
aws_iam_policy" "adminuser" > policy
resource "aws_iam_user" "Admin-user" {
name = "luffy"
tags = {
"description" = "Technical Team Lead"
}
}
resource "aws_iam_policy" "adminuser" {
name = "AdminUsers"
policy = <<EOF
{
"Version": "2012-10-17",
"Statement": [
{
"Sid": "1234567890",
"Effect": "Allow",
"Action": "*",
"Resource": "*"
}
]
}
EOF
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
waseem:terraform-aws waseemakram$ terraform apply
aws_iam_user.Admin-user: Refreshing state... [id=luffy]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the follo
+ create
Terraform will perform the following actions:
# aws_iam_policy.adminuser will be created
+ resource "aws_iam_policy" "adminuser" {
+ arn = (known after apply)
+ attachment_count = (known after apply)
+ id = (known after apply)
+ name = "AdminUsers"
+ name_prefix = (known after apply)
+ path = "/"
+ policy = jsonencode(
{
+ Statement = [
+ {
+ Action = "*"
+ Effect = "Allow"
+ Resource = "*"
+ Sid = "1234567890"
},
]
+ Version = "2012-10-17"
}
)
+ policy_id = (known after apply)
+ tags_all = (known after apply)
}

```
  # aws_iam_user_policy_attachment.luffy-admin-access will be created
  + resource "aws_iam_user_policy_attachment" "luffy-admin-access" {
      + id         = (known after apply)
      + policy_arn = (known after apply)
      + user       = "luffy"
    }

Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_iam_policy.adminuser: Creating...
aws_iam_policy.adminuser: Creation complete after 2s [id=arn:aws:iam::207662791773:policy/AdminUsers]
aws_iam_user_policy_attachment.luffy-admin-access: Creating...
aws_iam_user_policy_attachment.luffy-admin-access: Creation complete after 1s [id=luffy-20260119092823547600000001]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
waseem:terraform-aws waseemakram$
```

—-> now check the aws account and check the policy which we added to user named luffy

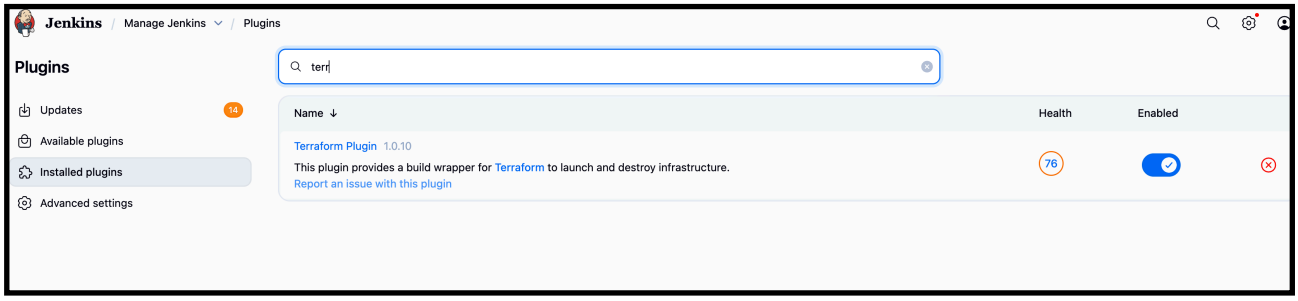


3.Integrate Terraform in Jenkins using the Terraform plugin.

Step 1: Install the Terraform Plugin in Jenkins

1. Go to **Jenkins Dashboard → Manage Jenkins → Plugins → Available Plugins**.
2. Search for **Terraform**.
3. Install **Terraform Plugin**
4. Restart Jenkins if required.

—-> we are creating a user named zoro in our aws account with this task using Jenkins pipeline

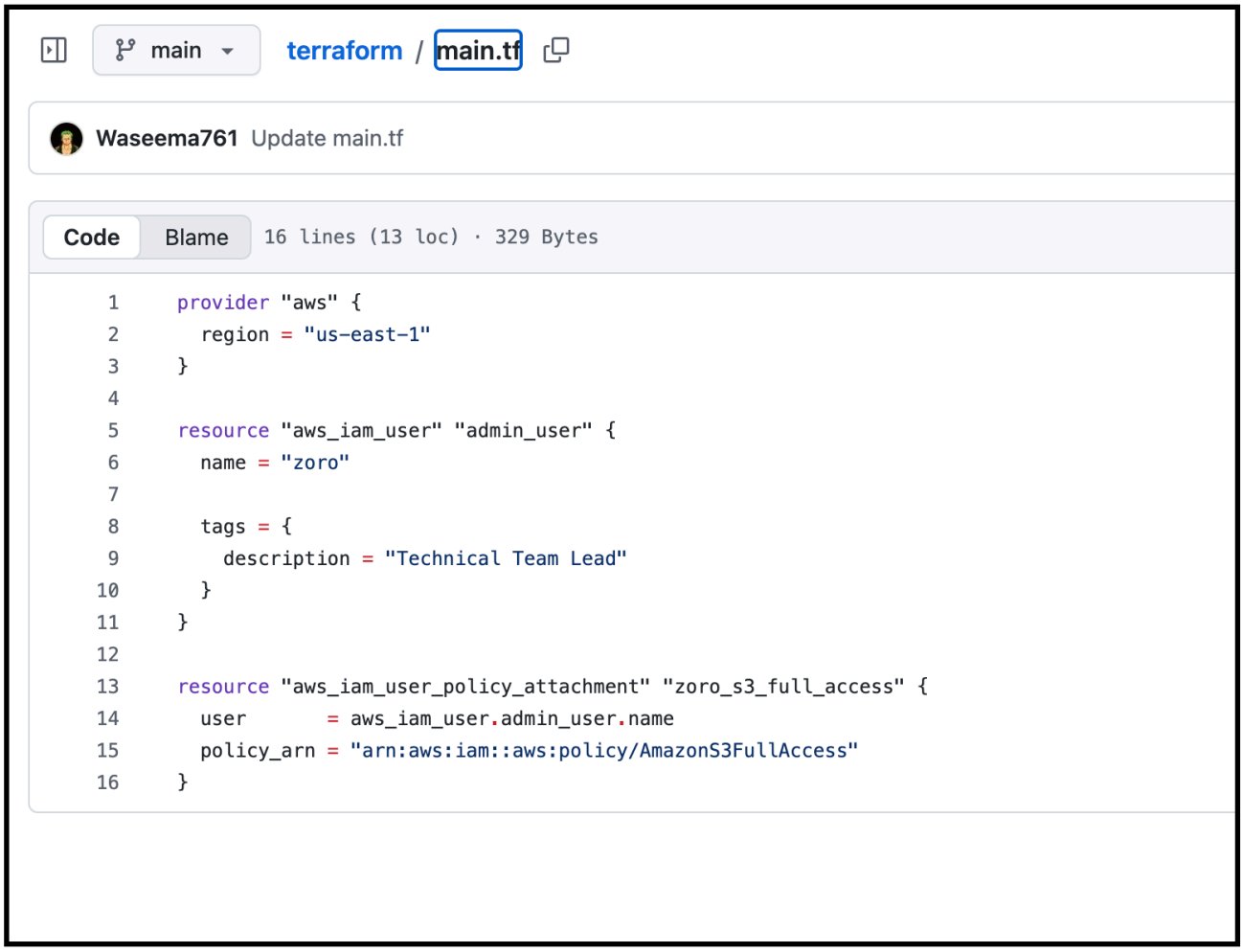


Step 2: Create a Jenkins Job (Pipeline or Freestyle)

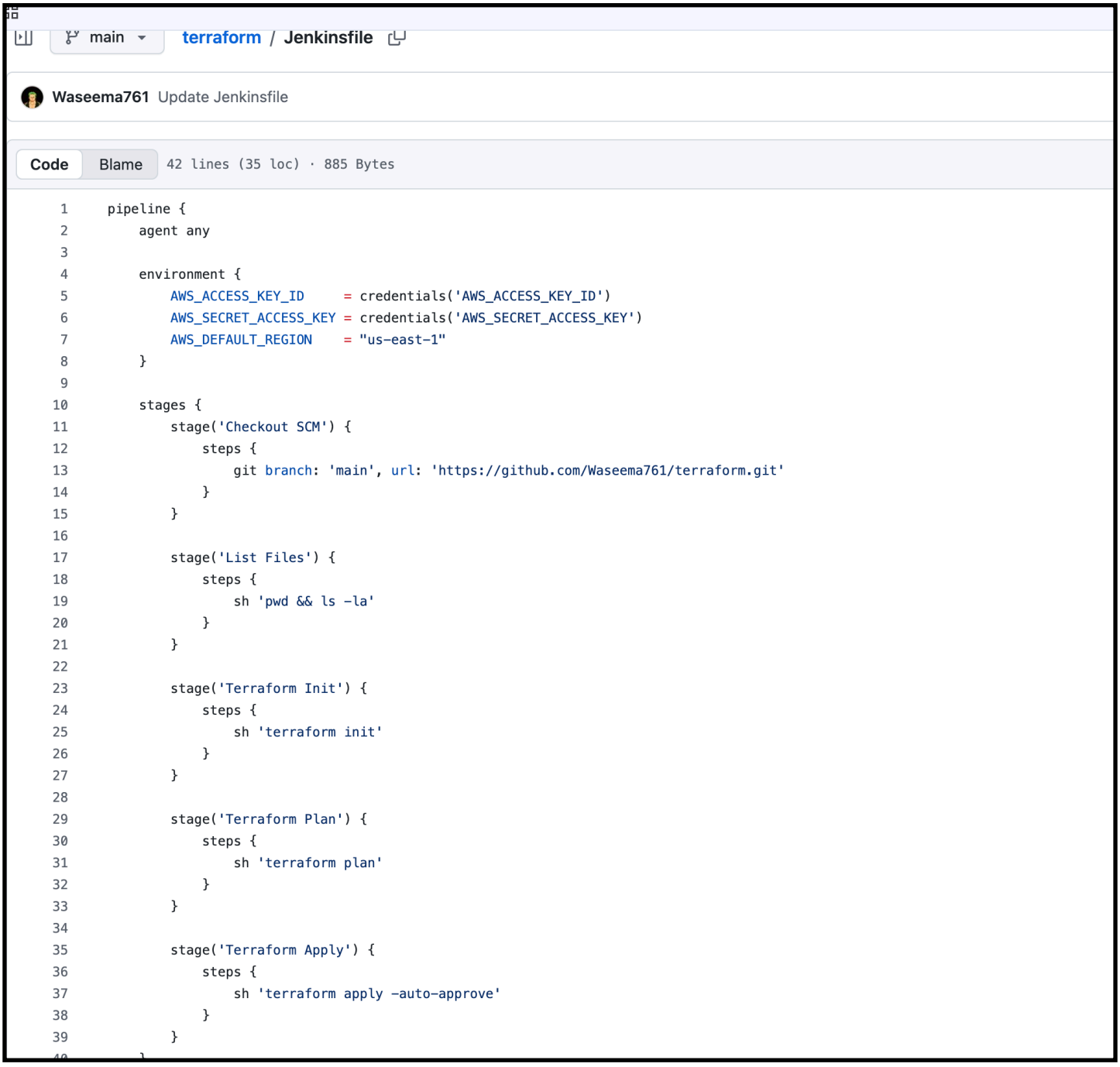
You can use either **Freestyle** or **Pipeline**.

—> inside GitHub

—> add this code in main.tf in your repo file



jenkinsfile



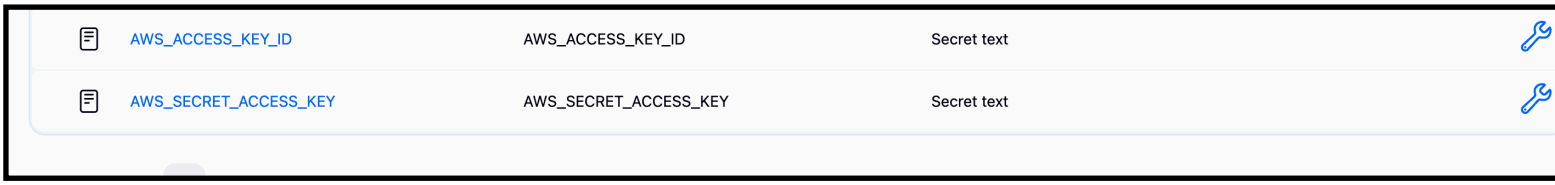
main terraform / Jenkinsfile

Waseema761 Update Jenkinsfile

Code Blame 42 lines (35 loc) · 885 Bytes

```
pipeline {
    agent any

    environment {
        AWS_ACCESS_KEY_ID     = credentials('AWS_ACCESS_KEY_ID')
        AWS_SECRET_ACCESS_KEY = credentials('AWS_SECRET_ACCESS_KEY')
        AWS_DEFAULT_REGION    = "us-east-1"
    }

    stages {
        stage('Checkout SCM') {
            steps {
                git branch: 'main', url: 'https://github.com/Waseema761/terraform.git'
            }
        }

        stage('List Files') {
            steps {
                sh 'pwd && ls -la'
            }
        }

        stage('Terraform Init') {
            steps {
                sh 'terraform init'
            }
        }

        stage('Terraform Plan') {
            steps {
                sh 'terraform plan'
            }
        }

        stage('Terraform Apply') {
            steps {
                sh 'terraform apply -auto-approve'
            }
        }
    }
```

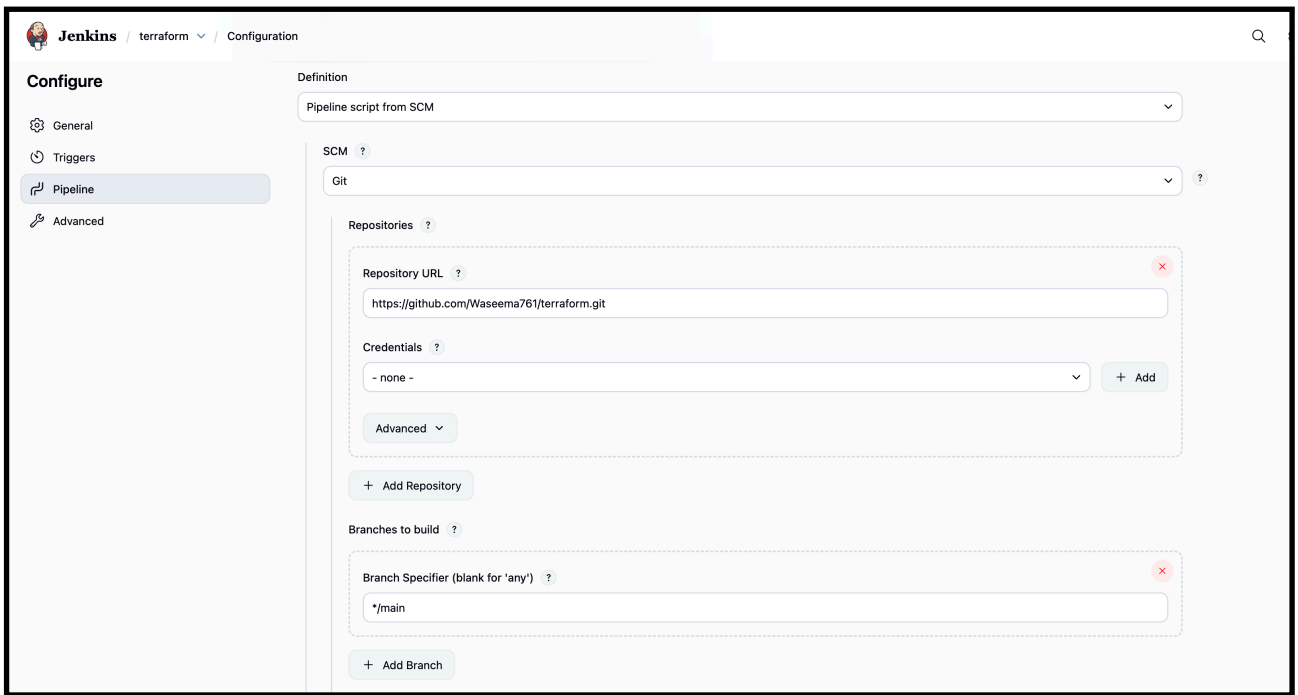
—> make sure you created credentials of your aws access key details in Jenkins gui credentials



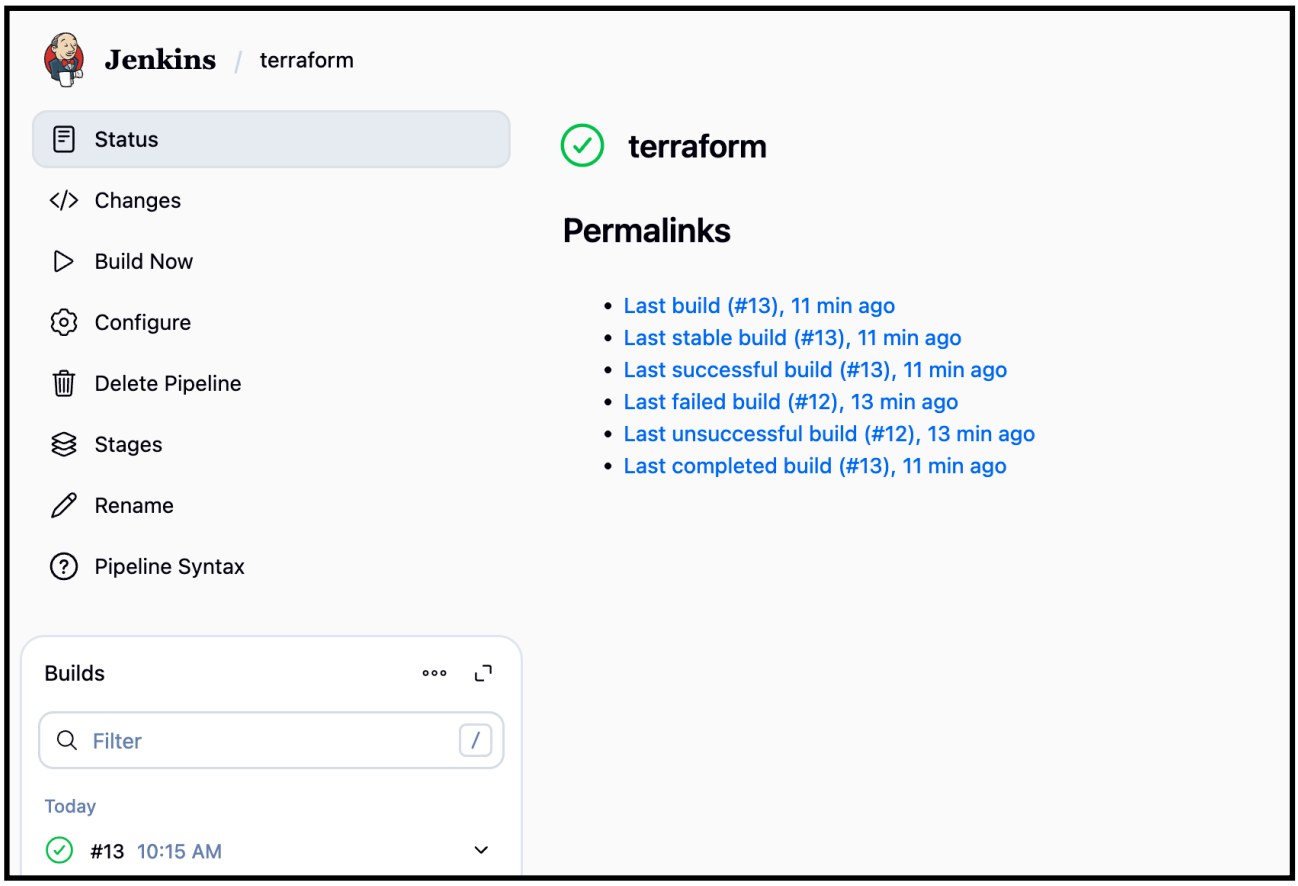
—> access key id : your access key id attach it secret text
—-> secret access key : your aws access secret key attach it with secret text credential

Now create a job and build :

—> attach your repo were we added jenkinsfile and main.tf



—> build job :

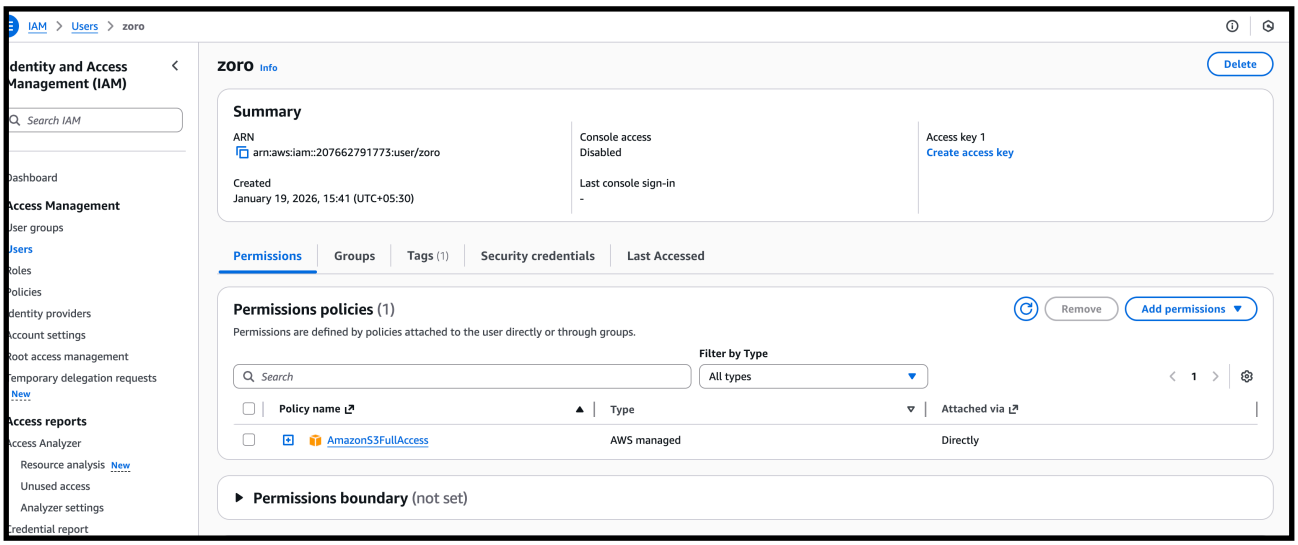


—> check the console output:

```
Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
   [32m+ [0m create [0m

Terraform will perform the following actions:

 [1m  # aws_iam_user_policy_attachment.zoro_s3_full_access [0m will be created
 [0m   [32m+ [0m [0m resource "aws_iam_user_policy_attachment" "zoro_s3_full_access" {
       [32m+ [0m [0m id         = (known after apply)
       [32m+ [0m [0m policy_arn = "arn:aws:iam::aws:policy/AmazonS3FullAccess"
       [32m+ [0m [0m user       = "zoro"
     }

 [1mPlan: [0m  [0m1 to add, 0 to change, 0 to destroy.
 [0m [1maws_iam_user_policy_attachment.zoro_s3_full_access: Creating... [0m [0m
 [0m [1maws_iam_user_policy_attachment.zoro_s3_full_access: Creation complete after 0s [id=zoro-20260119101544320600000001] [0m
 [0m [1m [32m
Apply complete! Resources: 1 added, 0 changed, 0 destroyed. [0m
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

—-> pipeline success means we have successfully created a iam user named zoro in our aws account with least permission policy

—-> check your aws account —> iam —> users



—-> are able to see the policy which we have given to user is available here

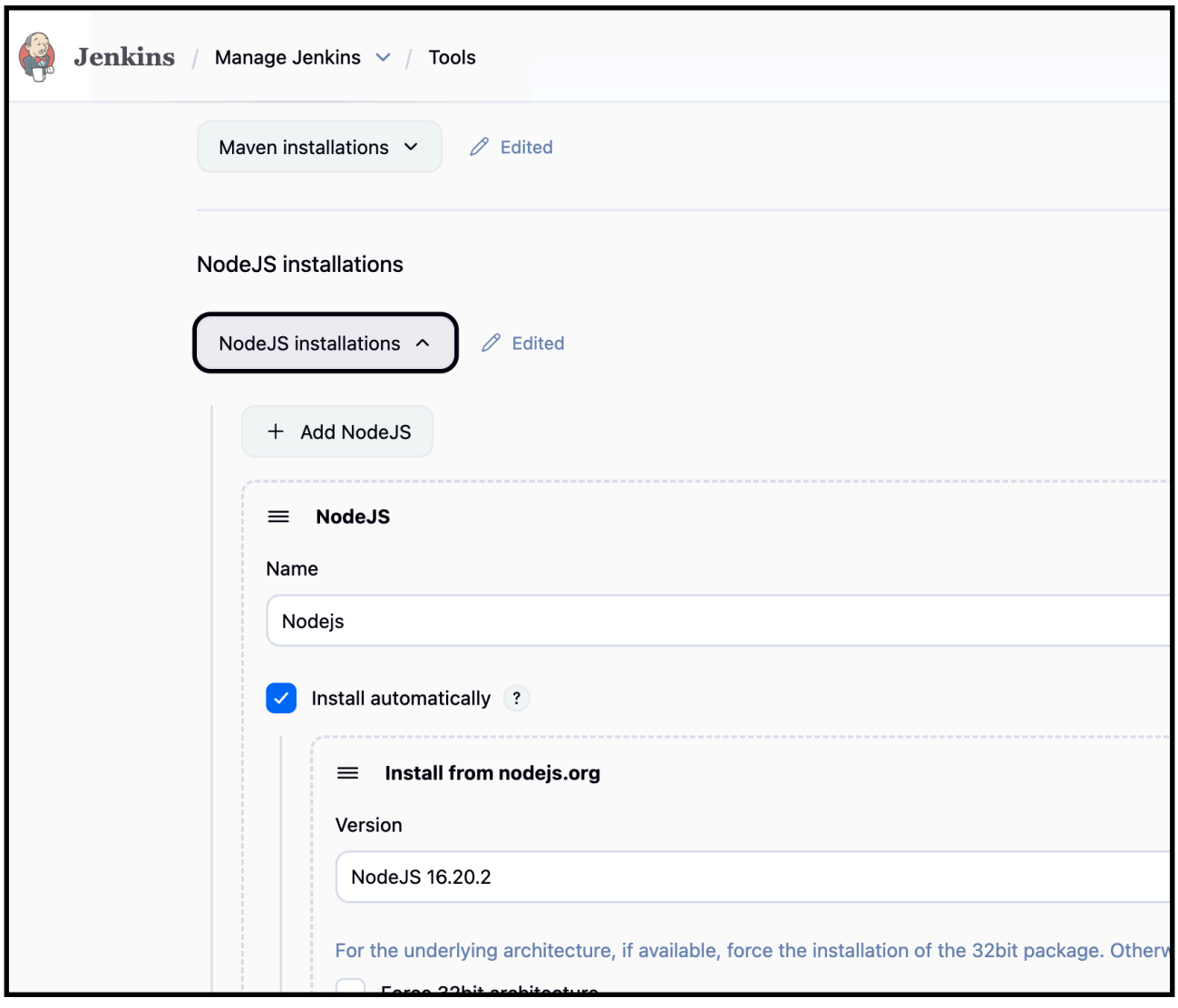
4. Create a CI/CD pipeline for a Nodejs Application: https://github.com/betawins/Trading-UI.git

Set Up Jenkins

1. Install required plugins:
    - **NodeJS plugin** (to install Node.js runtime)

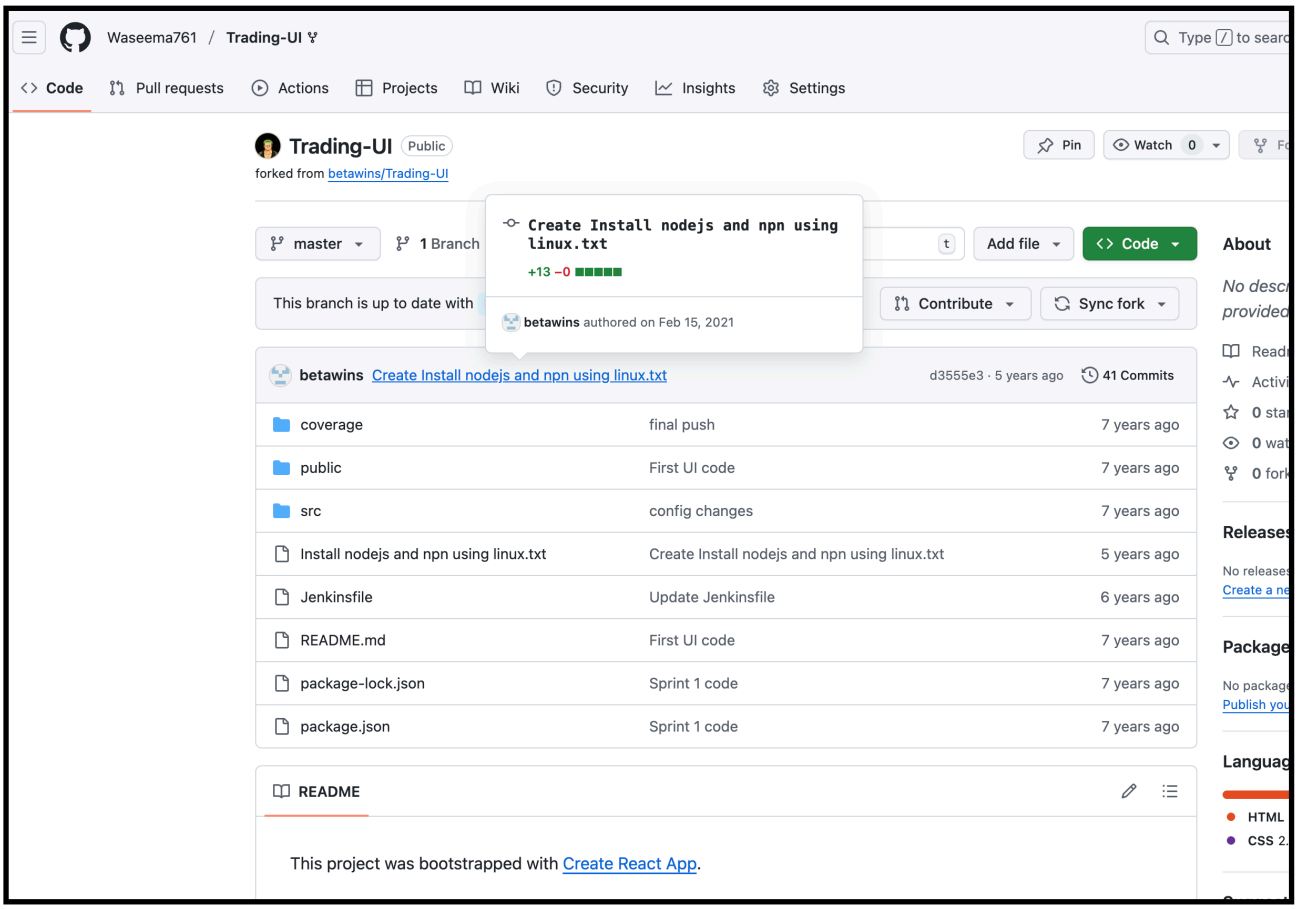
Configure **NodeJS Tool** in Jenkins:

- Go to **Manage Jenkins → Global Tool Configuration → NodeJS installations**
- Add a version (e.g., NodeJS 16.x).



Fork the github repo of the url

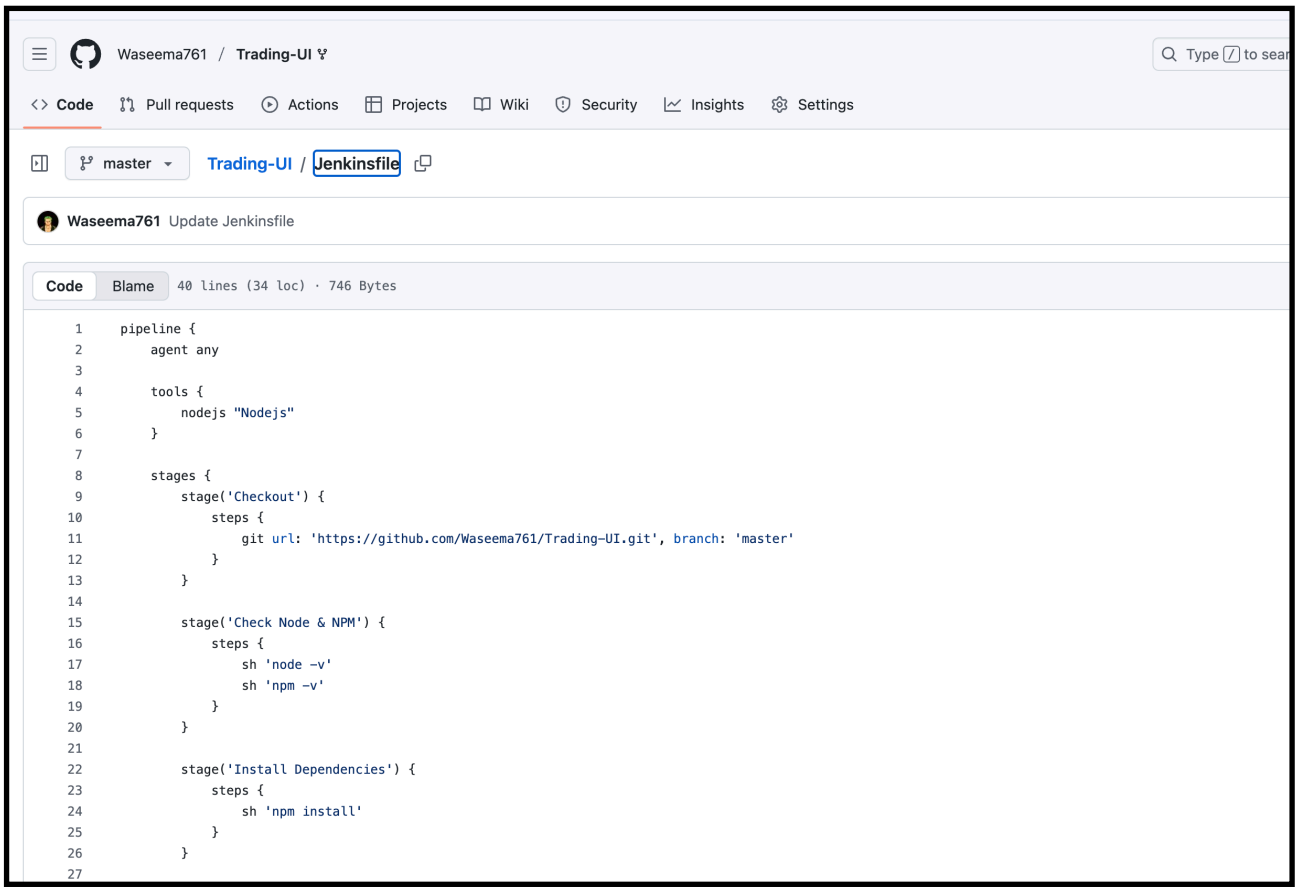
. Create Jenkinsfile in GitHub Repo



—> Inside jenkinsfile

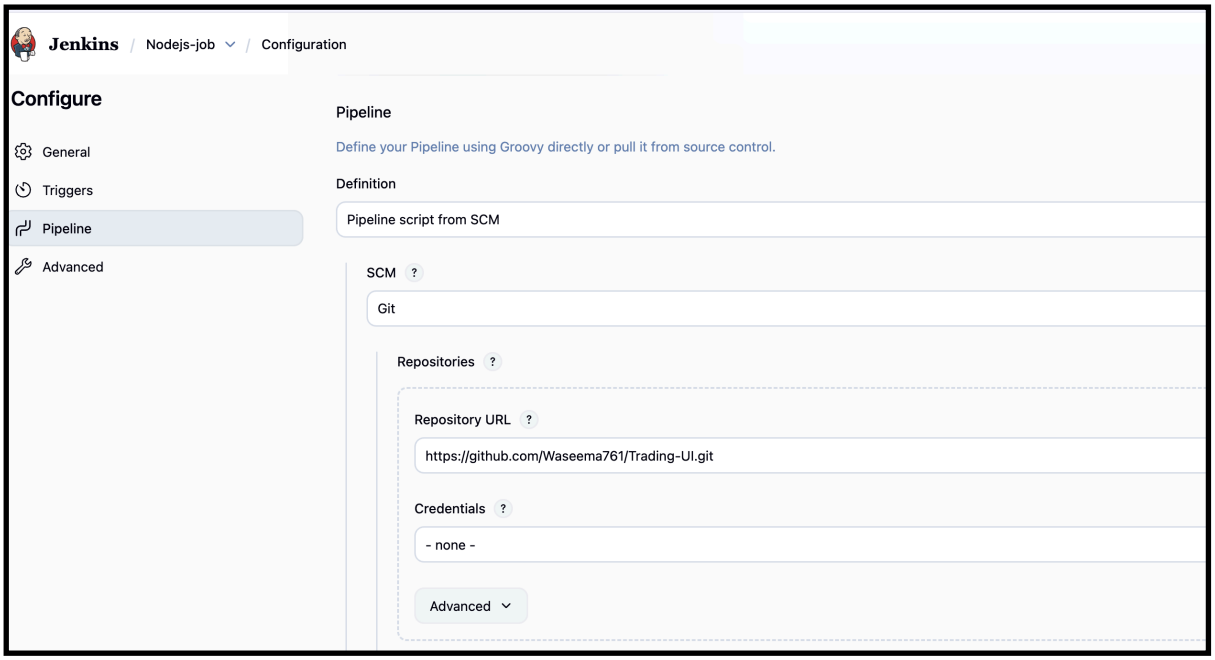
```
pipeline {
    agent any

    tools {
        nodejs "Nodejs"
    }

    stages {
        stage('Checkout') {
            steps {
                git url: 'https://github.com/Waseema761/Trading-UI.git',
branch: 'master'
            }
        }

        stage('Check Node & NPM') {
            steps {
                sh 'node -v'
                sh 'npm -v'
            }
        }

        stage('Install Dependencies') {
            steps {
                sh 'npm install'
            }
        }

        stage('Build') {
            steps {
                sh 'CI=false npm run build'
            }
        }

        stage('Test') {
            steps {
                sh 'npm test || echo "No tests found"'
            }
        }
    }
```

}

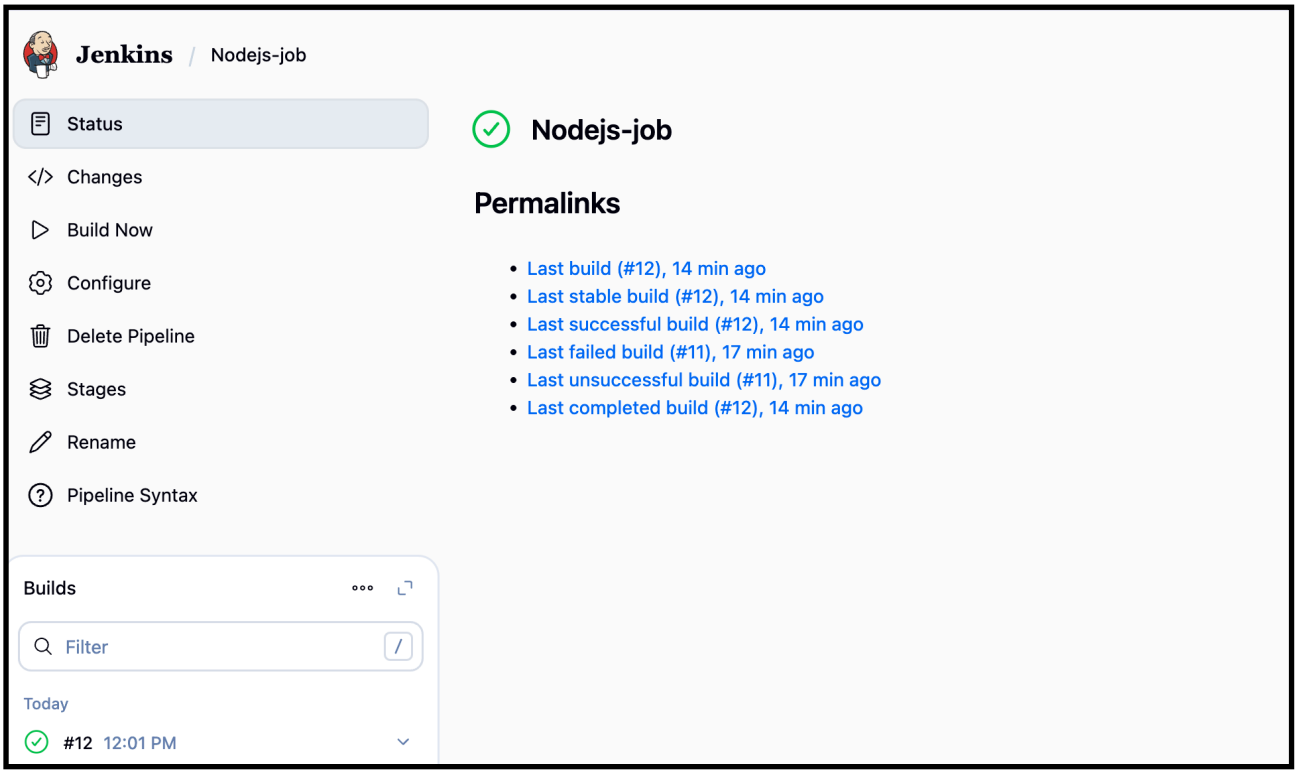


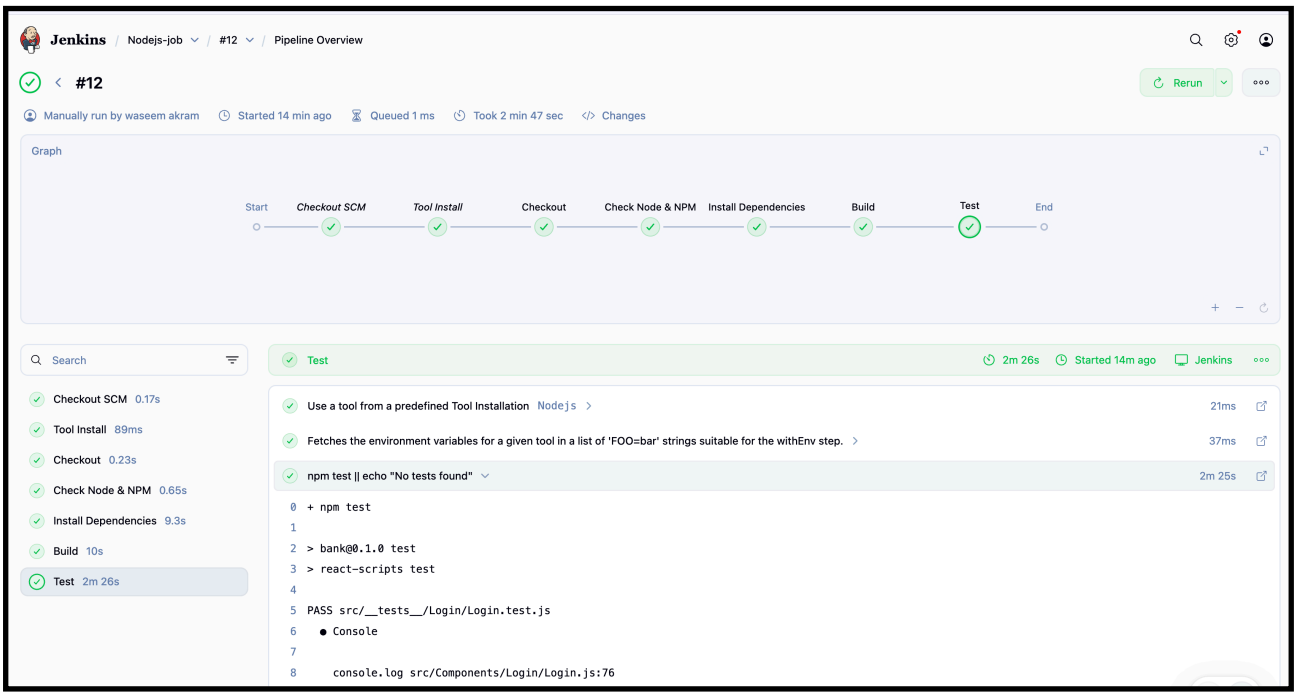
—-> now create a Job and give name Nodejs-job
—-> attach the repo were we edited our jenkinsfile

- New Item → Pipeline → Name it → Select "Pipeline from SCM"
- Choose **Git** and provide repo URL.

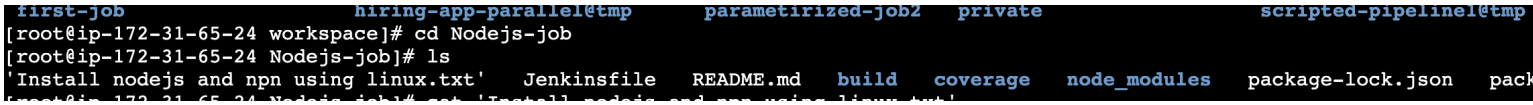


Save and build





—> check cli of Jenkins workspace



5.) Explain 10 Maven commands.

1. mvn clean

- Removes the **target/** directory (where compiled files, packaged JARs/WARs, etc. are stored).
- Ensures you start with a **clean build environment**.

mvn clean

---

2. mvn compile

- Compiles the **source code** of the project (from src/main/java).
- Output goes to target/classes.

mvn compile

---

3. mvn test

- Compiles and runs the **unit tests** (in src/test/java) using JUnit/TestNG.

mvn test

---

4. mvn package

- Packages the compiled code into a **JAR/WAR** file as defined in pom.xml.
- Example: target/my-app-1.0-SNAPSHOT.jar

mvn package

---

5. mvn install

- Installs the packaged project into the **local Maven repository** (~/.m2/repository).
- Makes the artifact available for use in other local projects.

mvn install

---

6. mvn deploy

- Copies the packaged code to a **remote repository** (like Nexus, Artifactory).
- Used in CI/CD pipelines for sharing artifacts with other developers.

mvn deploy

---

7. mvn site

- Generates a **site documentation** (reports, dependencies, plugins info, etc.) for the project in target/site.

mvn site

---

8. mvn dependency:tree

- Shows the **dependency hierarchy** of the project.
- Useful for identifying conflicts between library versions.

mvn dependency:tree

---

9. mvn validate

- Validates the project's structure and checks if pom.xml is correct.
- Runs before compilation.

mvn validate

---

10. mvn verify

- Runs any **integration tests** and checks if the package is valid.
- Ensures the project is ready for deployment.

mvn verify

Summary:

Command	Purpose
mvn clean	Remove target/ build files
mvn compile	Compile source code
mvn test	Run unit tests
mvn package	Package into JAR/WAR
mvn install	Install into local repo
mvn deploy	Deploy to remote repo
mvn site	Generate project site reports
mvn dependency:tree	Show dependency tree
mvn validate	Validate project structure
mvn verify	Run integration tests & verify package